HEH.be
Sciences et technologies
8a, avenue V. Maistriau B-7000 Mons
Tél : +32 (0)65 33 81 54
Fax : +32 (0)65 31 30 51
E-mail : tech-mons@heh.be
www.heh.be

Report - Parallel Architecture

2024 - 2025

Lassois Patrick

Formations de Bachelliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer

Table des matières

1 Introduction 3
2 Hardware description 3
3 TP1 4
4 TP2 4
5 TP3 5
6 TP4 6
7 TP5 8
8 TP Additional 8
8.1 Optimisation 1 8
8.1.1 TP1 9
8.1.2 TP4(B) 9
8.1.3 TP5 10
9 Conclusion 10
10 Annexes 11
10.1 TP1 code 11
10.2 TP2 code 13
10.3 TP3 code 15
10.4 TP4(A) code 17
10.5 TP4(B) code 19
10.6 TP5 code 21
10.7 TP additionnel 23
10.7.1 TP1 23
10.7.2 TP4(B) 25
10.7.3 TP5 27

1 Introduction

As part of the Parallel Architecture course, we will study and implement different approaches for performing matrix computations in C++ on both CPU and GPU.
The objective is to explore several methods to accomplish the same task in a faster and more efficient way. To achieve this, we will implement various techniques, such as using threads per element, threads per row, and performing computations on the GPU using CUDA.
All tests will be conducted on square matrices of sizes 100×100, 500×500, 1000×1000, 1500×1500, and 2000×2000.
We will measure the computation times for each method, as well as additional metrics such as memory footprint and CPU/GPU usage percentage (only for matrices of size 1000×1000).

2 Hardware description

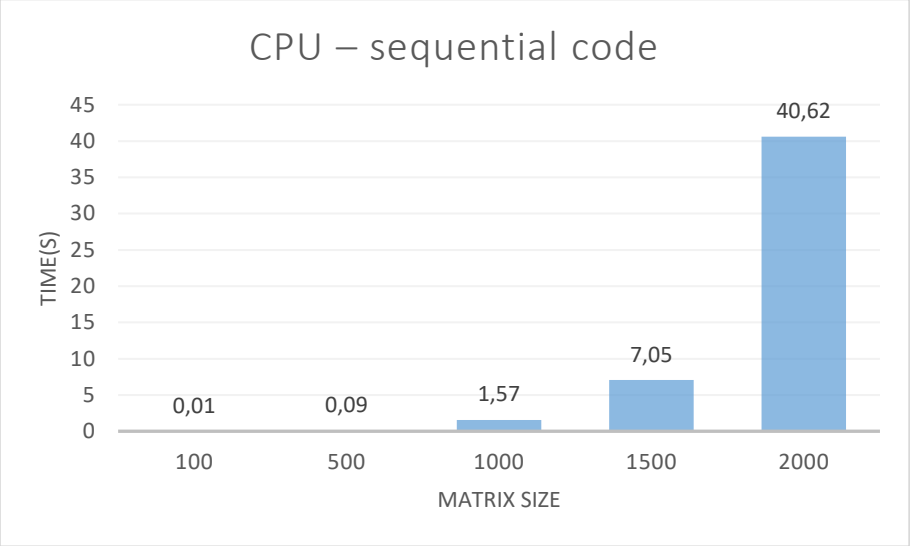
The tests were performed out on a PC with the following configuration:

- Processor: AMD Ryzen 5 4500U (6 cores) ;
- Graphics card: Integrated AMD Radeon Graphics ;
- Memory: 16 Go.

All the tests were performed on this PC, except for the one that required the GPU implementation using CUDA. CUDA cannot be executed on an AMD graphics card and only works with an NVIDIA card, so these tests were carried out on the school's computers.

3 TP1

In this first TP, we will test and implement a sequential code for matric multiplication. This code will then be used as a basic for improvement by adding varius optimisations to reduce execution time.
We obtain the following results:



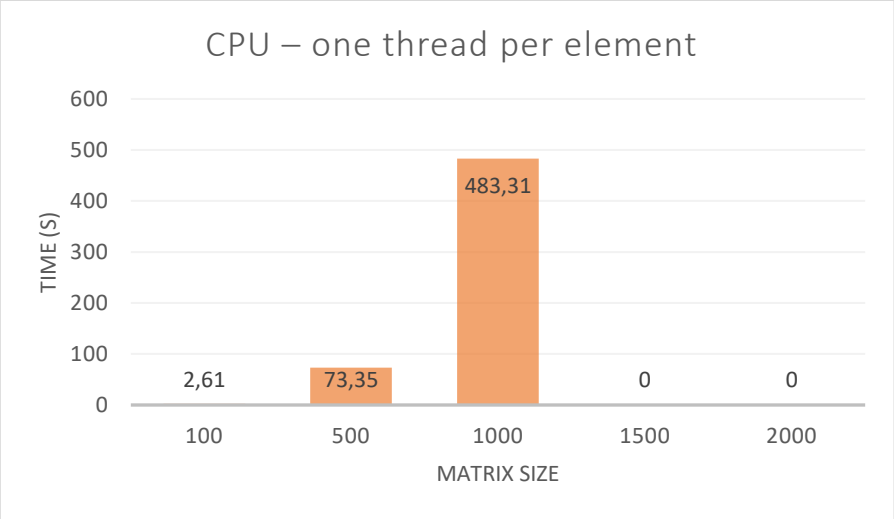
Additional measurements for the 1000x1000:

- CPU usage : 17% ;
- Memory : 14 Mo.

We can see that as the size of the matrix increases, so does the calculation time. This first simple technique therefore quickly becomes long and costly in terms of resources for large matrices. The results show the limitations of this method and demonstrate the need to explore other solutions.

4 TP2

In this second TP, we used threads to implement an improved version of the previous code. We will start with a simple approach, assign one thread per element.
We obtain the following results:



Formations de Bachelliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer

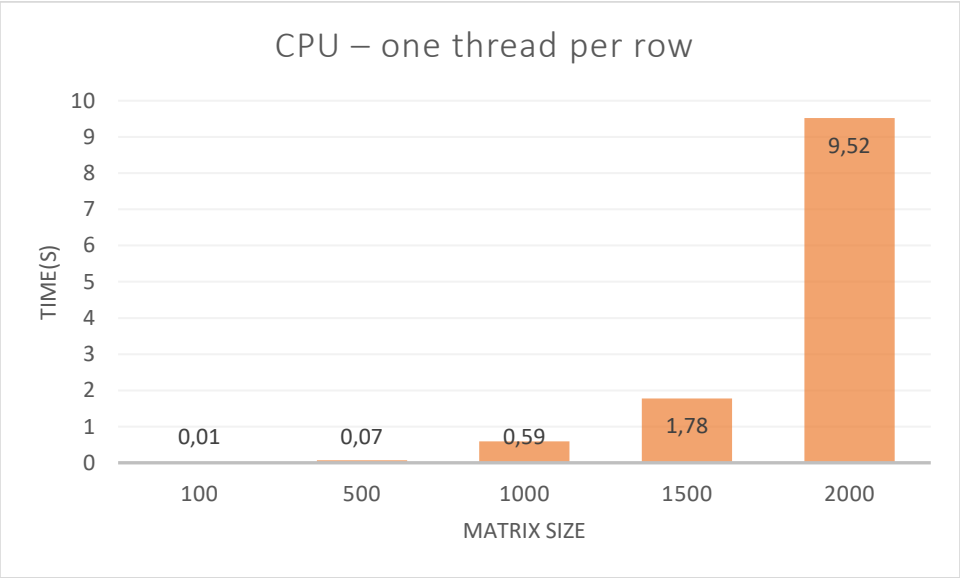
Additional measurements for the 1000x1000:

- CPU usage : 25% ;
- Memory : 59 Mo.

Setting up threads for matrix calculations can be very useful for performing matrix calculations simultaneously. However, it is important to organise their use properly. In this TP, we have associated one thread per element, but this is not an optimal approach, because for large matrices, it generates an excessive number of threads, which overloads the CPU and can cause the computer to crash, as we have seen with matrices of sizes 1500×1500 and 2000×2000.
Moreover, when execution does not cause a crash, the calculation times recorded are very high due to the high cost of creating and managing a large number of threads.

5 TP3

In this TP, we will implement the threads not by element but by row.
We obtain the following results:



Additional measurements for the 1000x1000:

- CPU usag : 64 % ;
- Memory : 40 Mo ;

This new approach is much more efficient than the previous one. Computation times are much lower than with the element approach, thanks to a reduced number of threads.

Formations de Bachelliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer

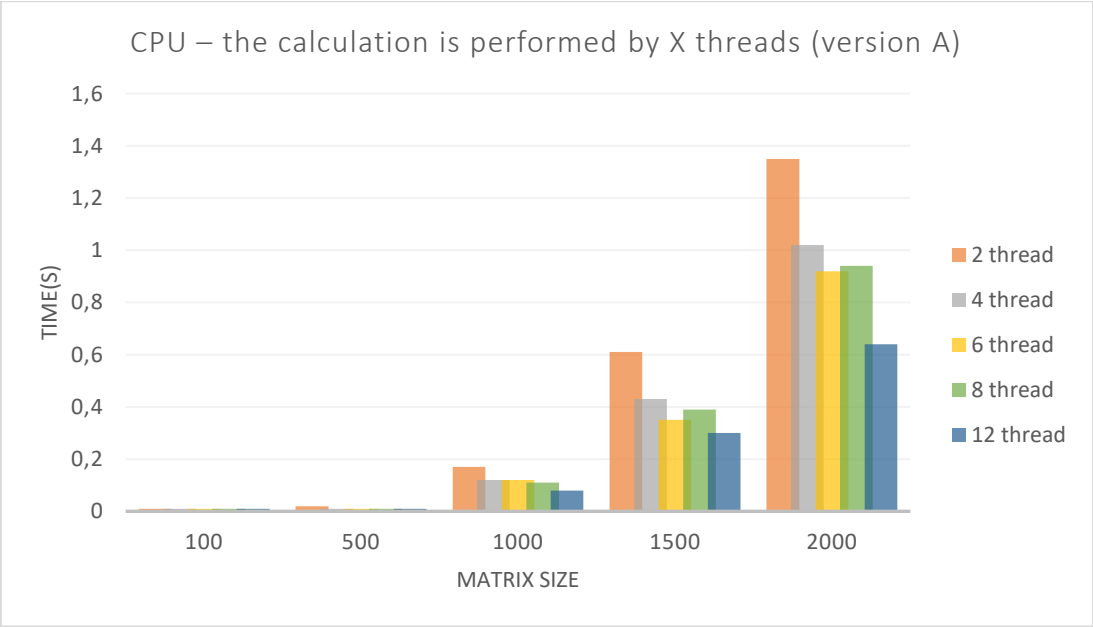
6 TP4

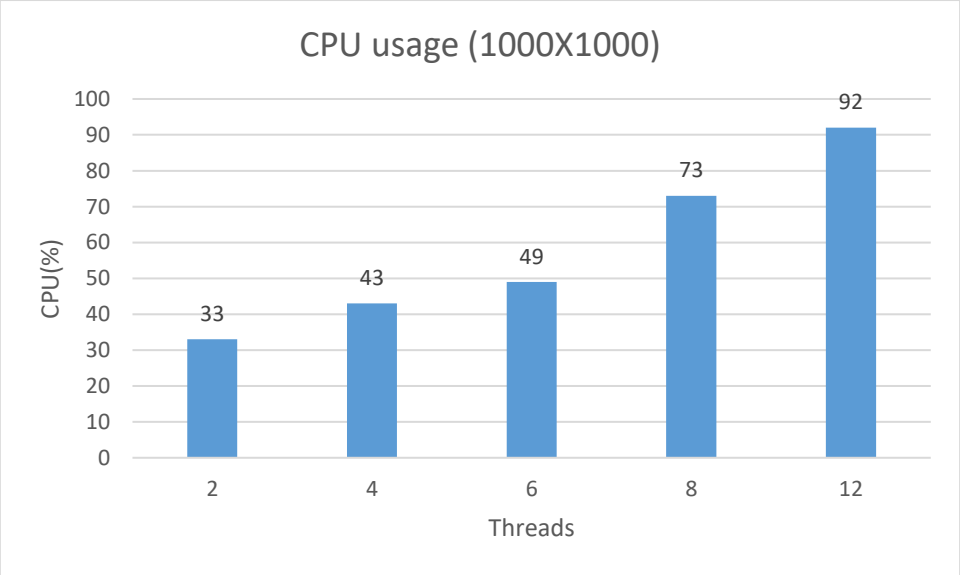
In this TP, we will implement an approach where calculations are performed by a number of x threads. Two different versions will be implemented:

- Version A with alternating thread distribution ;
- Version B with contiguous thread distribution.

Measurements were taken with 2, 4, 6, 8 and 12 threads.

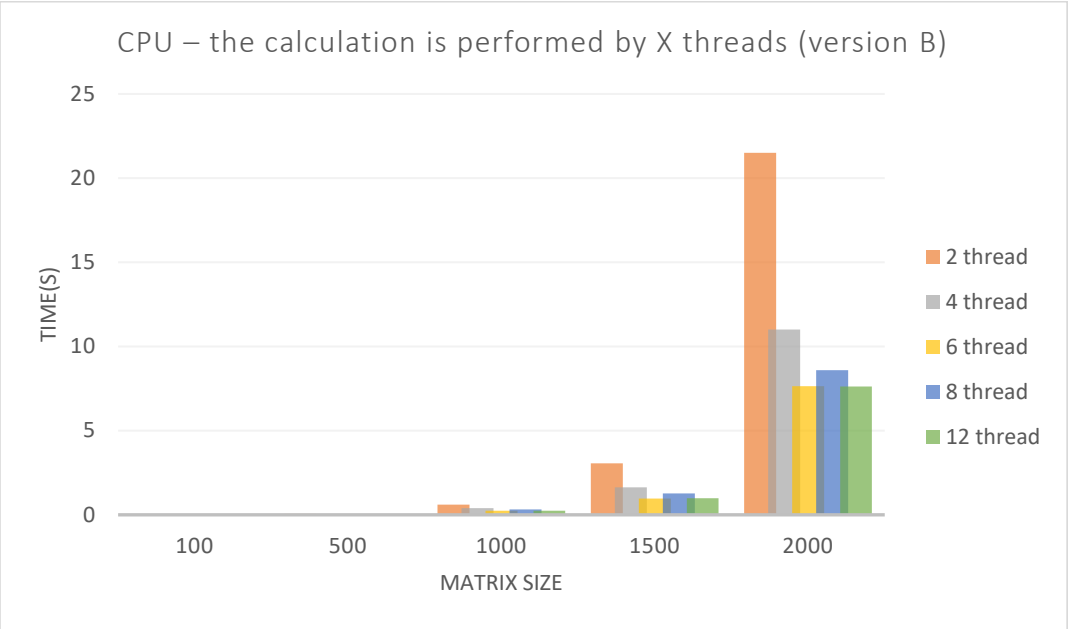
1. Version A

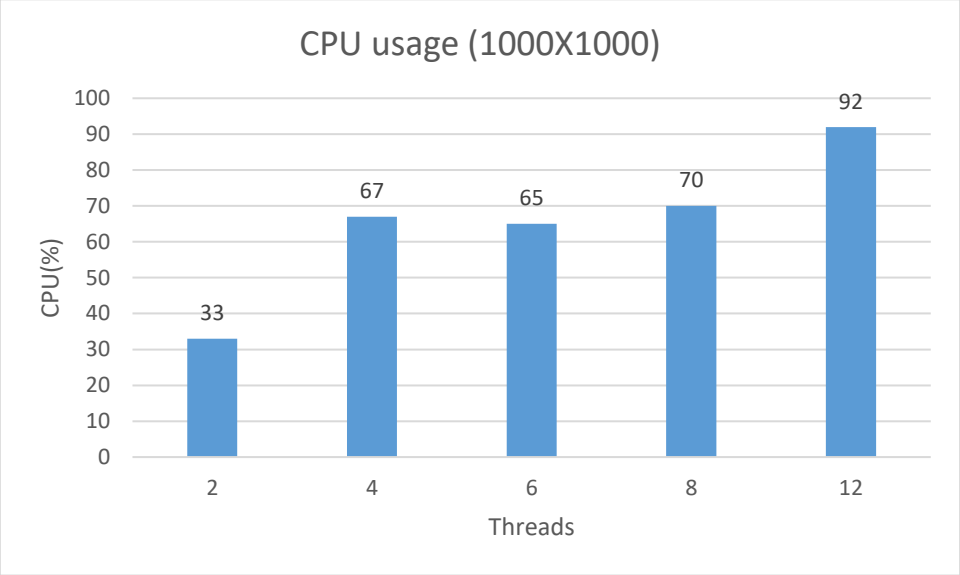




2. Version B

Formations de Bachelliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer



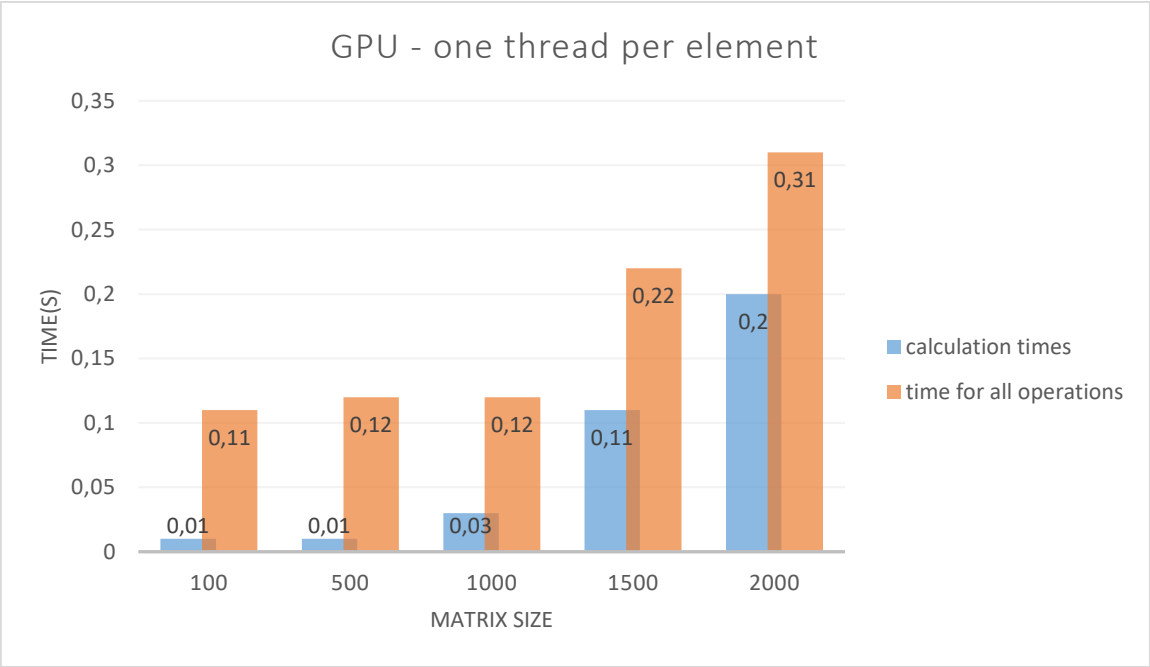


The graphs above illustrate two points:

- For both versions, as the number of threads increases, execution time decreases. This is due to a better distribution of the computing load between the threads.
- We can see that the alternating version performs better than the contiguous version. This is due to the way in which the threads access memory. In the alternating version, each thread processes spaced rows in the matrix, which distributes memory accesses more evenly. In contrast, the contiguous version assigns blocks of successive rows to each thread, slowing down execution.

7 TP5

In this TP, we will test the matrix calculation, not on the CPU, but on the GPU. The implementation will follow the same logic as TP2, one thread per element. However, in contrast to TP2 where we obtained very high execution times, we expect better performance here thanks to the parallel processing power of the GPU.
We obtain the following results:



Additional measurements for the 1000x1000:

- GPU usage : 1.2% ;
- CPU usage : 6% ;
- Memory : 96 Mo.

This last TP on GPU is really powerful, with very low calculation times.
This improvement is due to the architecture of the GPU, which is designed to carry out a very large number of operations in parallel, whereas the CPU is optimised for the rapid execution of a few threads.

8 TP Additional

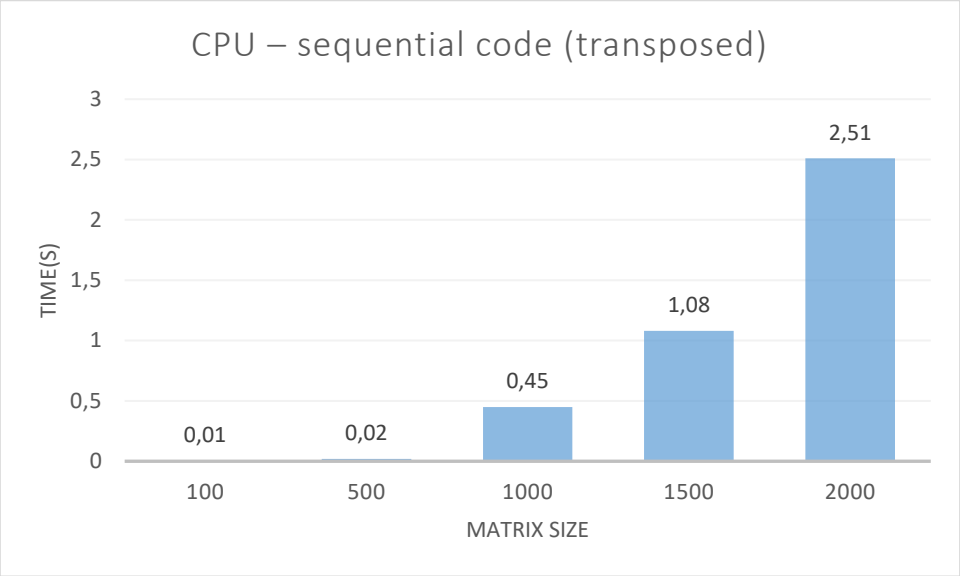
8.1 Optimisation 1

This optimisation consists of using the transposed B matrix to perform the calculation.

Formations de Bacheliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer

We observed lower execution times compared with the non-transposed version. This improvement can be explained by better memory access patterns, the transposing matrix B allows memory to be accessed sequentially by rows rather than columns, which aligns better with how data is stored in memory.

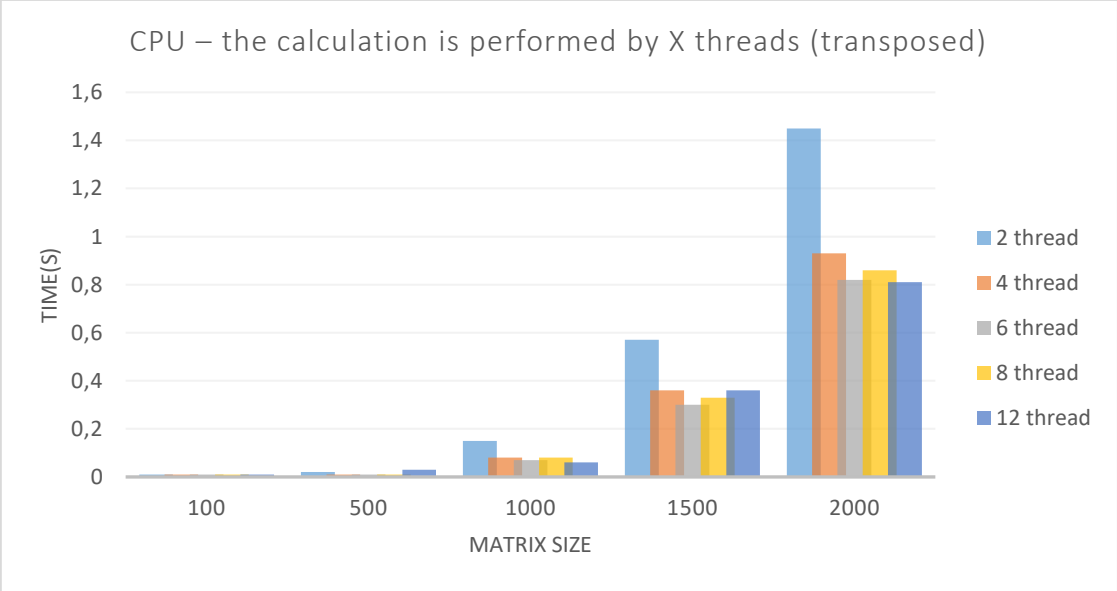
8.1.1 TP1



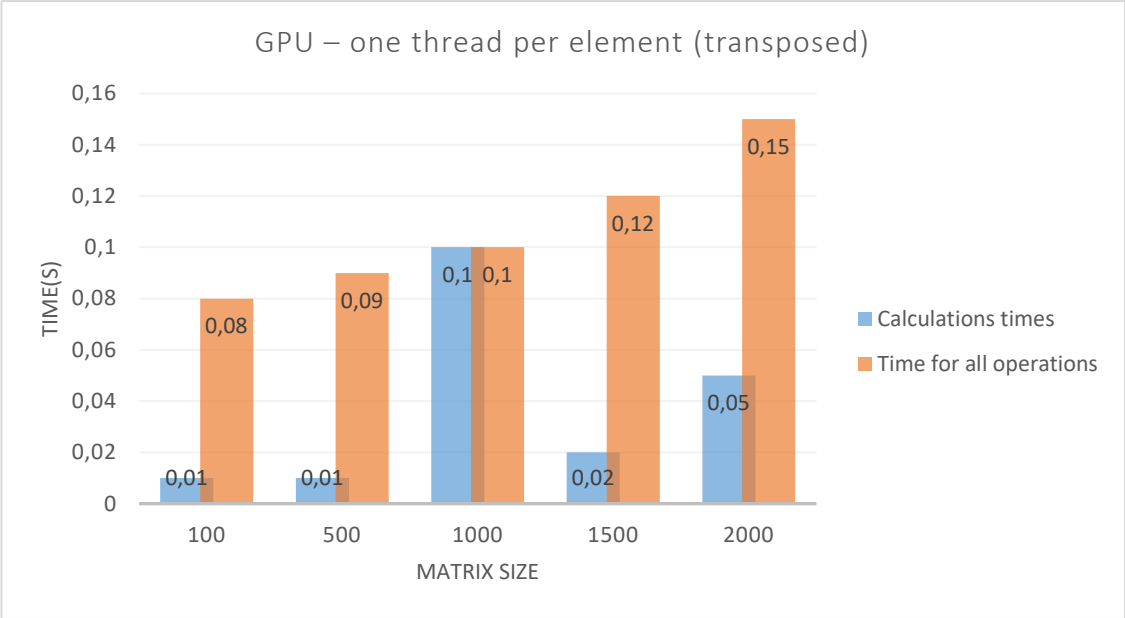
Additional measurements for the 1000x1000:

- CPU usage : 9% ;
- Memory : 19 Mo.

8.1.2 TP4(B)



8.1.3 TP5



Additional measurements for the 1000x1000:

- GPU usage : 1.1% ;
- CPU usage : 6% ;
- Memory : 88 Mo.

9 Conclusion

In conclusion, this work has allowed study of different approaches to matrix calculation on CPU and GPU, with the objective of optimizing performances.

By multithreading, we have been able to improve the performance of the basic sequential code, by parallelizing tasks and making the best use of available resources.

We were also able to compare the use of parallelism on CPU and GPU, where the GPU proved to be much faster and more suitable for executing a large number of threads concurrently.

Formations de Bachelliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer

10 Annexes

10.1 TP1 code

```
#include <iostream>
#include <ctime>
#include <chrono>
using namespace std;

int main() {

    int N = 0;
    cout << "Un nombre entier ? :";
    cin >> N;

    int** A = new int* [N];
    int** B = new int* [N];
    int** C = new int* [N];
    int somme = 0;

    srand(time(NULL));

    for (int X = 0; X < N; X++) {
        A[X] = new int[N];
        B[X] = new int[N];
        C[X] = new int[N];


        for (int L = 0; L < N; L++) {
            A[X][L] = rand() % 10 + 1;
            B[X][L] = rand() % 10 + 1;
            C[X][L] = 0;
        }
    }


    std::chrono::time_point<std::chrono::system_clock> start, end;

    start = std::chrono::system_clock::now();

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            int somme = 0;
            for (int k = 0; k < N; k++) {
                somme += A[i][k] * B[k][j];
            }
            C[i][j] = somme;
        }
```


Formations de Bacheliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer

```
    }

    end = std::chrono::system_clock::now();

    long long int microseconds = chrono::duration_cast<chrono::microseconds>(end -
start).count();

    cout << "Temps d'execution : " << microseconds << " micro sec\n";

    for (int i = 0; i < N; i++) {
        delete[] A[i];
        delete[] B[i];
        delete[] C[i];
    }
    delete[] A;
    delete[] B;
    delete[] C;

    return 0;
}
```

10.2 TP2 code

```
include <iostream>
#include <ctime>
#include <chrono>
#include <thread>

using namespace std;

void calcul(int N, int** A, int** B, int** C, int i, int j) {
    int somme = 0;
    for (int k = 0; k < N; k++) {
        somme += A[i][k] * B[k][j];
    }
    C[i][j] = somme;
}

int main() {
    int N;
    cout << "Un nombre entier ? : ";
    cin >> N;


    int** A = new int* [N];
    int** B = new int* [N];
    int** C = new int* [N];

    srand(time(0));

    for (int i = 0; i < N; i++) {
        A[i] = new int[N]();
        B[i] = new int[N]();
        C[i] = new int[N]();

        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 10 + 1;
            B[i][j] = rand() % 10 + 1;
        }
    }

    chrono::time_point<chrono::system_clock> start, end;
    start = chrono::system_clock::now();

    thread** threads = new thread*[N*N];
    int index = 0;

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
```

```
            threads[index] = new thread(calcul, N, A, B, C, i, j);

            index++;
        }
    }

    for (int i = 0; i < N * N; i++) {
        threads[i]->join();
    }


    delete[] threads;

    end = chrono::system_clock::now();
    long long microseconds = chrono::duration_cast<chrono::microseconds>(end -
start).count();

    cout << "Temps d'execution : " << microseconds << " microsec\n";


    for (int i = 0; i < N; i++) {
        delete[] A[i];
        delete[] B[i];
        delete[] C[i];
    }
    delete[] A;
    delete[] B;
    delete[] C;

    return 0;
}
```

10.3 TP3 code

```
#include <iostream>
#include <ctime>
#include <chrono>
#include <thread>

using namespace std;



void calcul(int N, int** A, int** B, int** C, int i) {
    for (int j = 0; j < N; j++) {
        int somme = 0;
        for (int k = 0; k < N; k++) {
            somme += A[i][k] * B[k][j];
        }
        C[i][j] = somme;
    }
}

int main() {
    int N;
    cout << "Un nombre entier ? : ";
    cin >> N;


    int** A = new int* [N];
    int** B = new int* [N];
    int** C = new int* [N];

    srand(time(0));

    for (int i = 0; i < N; i++) {
        A[i] = new int[N]();
        B[i] = new int[N]();
        C[i] = new int[N]();

        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 10 + 1;
            B[i][j] = rand() % 10 + 1;
        }
    }

    chrono::time_point<chrono::system_clock> start, end;
    start = chrono::system_clock::now();

    thread* threads = new thread [N];
```


Formations de Bachelliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer

```
    for (int i = 0; i < N; i++) {
            threads[i] = thread(calcul, N, A, B, C, i);
    }

    for (int i = 0; i < N; i++) {
        threads[i].join();

    }


    delete[] threads;

    end = chrono::system_clock::now();
    long long microseconds = chrono::duration_cast<chrono::microseconds>(end -
start).count();

    cout << "Temps d'execution : " << microseconds << " microsec\n";

    for (int i = 0; i < N; i++) {
        delete[] A[i];
        delete[] B[i];
        delete[] C[i];
    }
    delete[] A;
    delete[] B;
    delete[] C;

    return 0;
}
```

10.4 TP4(A) code

```
#include <iostream>
#include <ctime>
#include <chrono>
#include <thread>

using namespace std;

void calcul(int N, int** A, int** B, int** C, int id, int nbr_thread) {
    for (int i = id; i < N; i += nbr_thread) {
        for (int j = 0; j < N; ++j) {
            int somme = 0;
            for (int k = 0; k < N; k++) {
                somme += A[i][k] * B[j][k];
            }
            C[i][j] = somme;
        }
    }
}



int main() {
    int N = 0;
    int nbr_thread = 0;

    cout << "Un nombre entier ? :";
    cin >> N;
    cout << "Un nombre de thread ?";
    cin >> nbr_thread;

    int** A = new int* [N];
    int** B = new int* [N];
    int** C = new int* [N];

    srand(time(NULL));

    for (int i = 0; i < N; i++) {
        A[i] = new int[N];
        B[i] = new int[N];
        C[i] = new int[N];

        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 10 + 1;
            B[i][j] = rand() % 10 + 1;
            C[i][j] = 0;
        }
```


Formations de Bachelliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer

```
    }

    std::chrono::time_point<std::chrono::system_clock> start, end;
    start = std::chrono::system_clock::now();

    thread* threads = new thread[nbr_thread];

    for (int i = 0; i < nbr_thread; i++) {
        threads[i] = thread(calcul, N, A, B, C, i, nbr_thread);
    }

    for (int i = 0; i < nbr_thread; ++i) {
        threads[i].join();
    }

    end = std::chrono::system_clock::now();
    long long int microseconds = chrono::duration_cast<chrono::microseconds>(end -
start).count();

    cout << "Temps d'execution : " << microseconds << " micro sec\n";


    for (int i = 0; i < N; i++) {
        delete[] A[i];
        delete[] B[i];
        delete[] C[i];
    }
    delete[] A;
    delete[] B;
    delete[] C;

    return 0;
}
```

10.5 TP4(B) code

```
#include <iostream>
#include <ctime>
#include <chrono>
#include <thread>

using namespace std;

void calcul(int N, int s, int** A, int** B, int** C, int id, int nbr_thread, int
start_idx, int end_idx) {
    for (int i = start_idx; i < end_idx; ++i) {
            for (int j = 0; j < N; ++j) {
                int somme = 0;
                for (int k = 0; k < N; k++) {
                    somme += A[i][k] * B[k][j];
                }
                C[i][j] = somme;
            }
    }
}

int main() {
    int N = 0;
    int nbr_thread = 0;

    cout << "Un nombre entier ? :";
    cin >> N;
    cout << "Un nombre de thread ?";
    cin >> nbr_thread;

    int** A = new int* [N];
    int** B = new int* [N];
    int** C = new int* [N];

    srand(time(NULL));

    for (int i = 0; i < N; i++) {
        A[i] = new int[N];
        B[i] = new int[N];
        C[i] = new int[N];

        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 10 + 1;
            B[i][j] = rand() % 10 + 1;
            C[i][j] = 0;
        }
    }
```

```
    std::chrono::time_point<std::chrono::system_clock> start, end;
    start = std::chrono::system_clock::now();

    thread* threads = new thread[nbr_thread];
    int Size = N / nbr_thread;

    for (int i = 0; i < nbr_thread; i++) {
        int start_idx = i * Size;
        int end_idx = (i == nbr_thread - 1) ? N : (i + 1) * Size;
        threads[i] = thread(calcul, N,i, A, B, C, i, nbr_thread, start_idx,
end_idx);
    }

    for (int i = 0; i < nbr_thread; i++) {
        threads[i].join();
    }

    end = std::chrono::system_clock::now();
    long long int microseconds = chrono::duration_cast<chrono::microseconds>(end -
start).count();

    cout << "Temps d'execution : " << microseconds << " micro sec\n";

    for (int i = 0; i < N; i++) {
        delete[] A[i];
        delete[] B[i];
        delete[] C[i];
    }
    delete[] A;
    delete[] B;
    delete[] C;


    return 0;
}
```

10.6 TP5 code

```
#include <iostream>
#include <ctime>
#include <chrono>
#include <cuda_runtime.h>

using namespace std;

__global__ void calcul(int* A, int* B, int* C, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int somme = 0;

    if (i < N * N)
    {
        int row = i / N;
        int col = i % N;
        for (int k = 0; k < N; k++)
        {
            somme += A[row * N + k] * B[k * N + col];
        }
        C[i] = somme;
    }

}

int main() {
    int N = 0;
    int nbr_thread = 256;

    cout << "Un nombre entier ? :";
    cin >> N;


    int* A = new int[N * N];
    int* B = new int[N * N];
    int* C = new int[N * N];
    int size = N * N * sizeof(int);


    for (int i = 0; i < N * N; i++) {
        A[i] = rand() % 10 + 1;
        B[i] = rand() % 10 + 1;
    }

    std::chrono::time_point<std::chrono::system_clock> start, end;
    start = std::chrono::system_clock::now();
```


Formations de Bachelliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer

```
    int* MGA, * MGB, * MGC;
    cudaMalloc((void**)&MGA, size);
    cudaMalloc((void**)&MGB, size);
    cudaMalloc((void**)&MGC, size);

    cudaMemcpy(MGA, A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(MGB, B, size, cudaMemcpyHostToDevice);

    dim3 threadsPerBlock(nbr_thread);
    dim3 numBlocks((N * N + threadsPerBlock.x - 1) / threadsPerBlock.x);

    std::chrono::time_point<std::chrono::system_clock> start2, end2;
    start2 = std::chrono::system_clock::now();

    calcul <<<numBlocks, threadsPerBlock >>> (MGA, MGB, MGC, N);

    cudaDeviceSynchronize();

    cudaMemcpy(C, MGC, size, cudaMemcpyDeviceToHost);

    end2 = std::chrono::system_clock::now();
    long long int microseconds2 = chrono::duration_cast<chrono::microseconds>(end2
- start2).count();

    end = std::chrono::system_clock::now();
    long long int microseconds = chrono::duration_cast<chrono::microseconds>(end -
start).count();


    cout << "Temps d'execution tout : " << microseconds << " micro sec\n";
    cout << "Temps d'execution calcul: " << microseconds2 << " micro sec\n";

    cudaFree(MGA);
    cudaFree(MGB);
    cudaFree(MGC);

    delete[] A;
    delete[] B;
    delete[] C;

    return 0;
}
```

10.7 TP additionnel

10.7.1 TP1

```
#include <iostream>
#include <ctime>
#include <chrono>
using namespace std;

int main() {

    int N = 0;
    cout << "Un nombre entier ? :";
    cin >> N;

    int** A = new int* [N];
    int** B = new int* [N];
    int** C = new int* [N];
    int somme = 0;
    int** B_transpo = new int* [N];

    srand(time(NULL));

    for (int i = 0; i < N; i++) {
        A[i] = new int[N];
        B[i] = new int[N];
        C[i] = new int[N];

        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 10 + 1;
            B[i][j] = rand() % 10 + 1;
            C[i][j] = 0;
        }
    }

    for (int X = 0; X < N; X++) {
        B_transpo[X] = new int[N];
        for (int L = 0; L < N; L++) {
            B_transpo[X][L] = B[L][X];
        }
    }


    std::chrono::time_point<std::chrono::system_clock> start, end;

    start = std::chrono::system_clock::now();

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
```


Formations de Bachelliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer

```
            int somme = 0;
            for (int k = 0; k < N; k++) {
                somme += A[i][k] * B_transpo[j][k];
            }
            C[i][j] = somme;
        }
    }

    end = std::chrono::system_clock::now();


    long long int microseconds = chrono::duration_cast<chrono::microseconds>(end -
start).count();

    cout << "Temps d'execution : " << microseconds << " micro sec\n";


    for (int i = 0; i < N; i++) {
        delete[] A[i];
        delete[] B[i];
        delete[] C[i];
        delete[] B_transpo[i];
    }
    delete[] A;
    delete[] B;
    delete[] C;
    delete[] B_transpo;

    return 0;
}
```

10.7.2 TP4(B)

```
#include <iostream>
#include <ctime>
#include <chrono>
#include <thread>
#include <mutex>

using namespace std;



void calcul(int N, int s, int** A, int** BT, int** C, int id, int nbr_thread, int
start_idx, int end_idx) {
    for (int i = start_idx; i < end_idx; ++i) {
        for (int j = 0; j < N; ++j) {
            int somme = 0;
            for (int k = 0; k < N; k++) {
                somme += A[i][k] * BT[j][k];
            }
            C[i][j] = somme;
        }
    }
}

int main() {
    int N = 0;
    int nbr_thread = 0;

    cout << "Un nombre entier ? :";
    cin >> N;
    cout << "Un nombre de thread ?";
    cin >> nbr_thread;

    int** A = new int* [N];
    int** B = new int* [N];
    int** C = new int* [N];
    int** B_transpo = new int* [N];

    srand(time(NULL));

    for (int i = 0; i < N; i++) {
        A[i] = new int[N];
        B[i] = new int[N];
        C[i] = new int[N];

        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 10 + 1;
            B[i][j] = rand() % 10 + 1;
```


Formations de Bachelliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer

```
            C[i][j] = 0;
        }
    }

    for (int X = 0; X < N; X++) {
        B_transpo[X] = new int[N];
        for (int L = 0; L < N; L++) {
            B_transpo[X][L] = B[L][X];
        }
    }

    std::chrono::time_point<std::chrono::system_clock> start, end;
    start = std::chrono::system_clock::now();

    thread* threads = new thread[nbr_thread];
    int chunkSize = N / nbr_thread;

    for (int i = 0; i < nbr_thread; i++) {
        int start_idx = i * chunkSize;
        int end_idx = (i == nbr_thread - 1) ? N : (i + 1) * chunkSize;
        threads[i] = thread(calcul, N, i, A, B_transpo, C, i, nbr_thread,
start_idx, end_idx);
    }

    for (int i = 0; i < nbr_thread; i++) {
        threads[i].join();
    }

    end = std::chrono::system_clock::now();
    long long int microseconds = chrono::duration_cast<chrono::microseconds>(end -
start).count();

    cout << "Temps d'execution : " << microseconds << " micro sec\n";

    for (int i = 0; i < N; i++) {
        delete[] A[i];
        delete[] B[i];
        delete[] C[i];
        delete[] B_transpo[i];
    }
    delete[] A;
    delete[] B;
    delete[] C;
    delete[] B_transpo;

    return 0;
}
```


Formations de Bacheliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer

10.7.3 TP5

```
#include <iostream>
#include <ctime>
#include <chrono>
#include <cuda_runtime.h>

using namespace std;

__global__ void calcul(int* A, int* B, int* C, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int somme = 0;

    if (i < N * N)
    {
        for (int k = 0; k < N; k++)
        {
            somme += A[k] * B[k];
        }
        C[i] = somme;
    }

}

int main() {
    int N = 0;
    int nbr_thread = 256;

    cout << "Un nombre entier ? :";
    cin >> N;


    int* A = new int[N * N];
    int** B = new int* [N];
    int* C = new int[N * N];
    int* B_transpo = new int[N * N];
    int size = N * N * sizeof(int);


    for (int i = 0; i < N * N; i++) {
        A[i] = rand() % 10 + 1;
        //B[i] = rand() % 10 + 1;
    }


    for (int i = 0; i < N; i++) {
        B[i] = new int[N];
        for (int j = 0; j < N; j++) {
            B[i][j] = rand() % 10 + 1;
```


Formations de Bachelliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer

```
        }
    }


    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            B_transpo[j * N + i] = B[i][j];
        }
    }

    std::chrono::time_point<std::chrono::system_clock> start, end;
    start = std::chrono::system_clock::now();

    int* MGA, * MGB, * MGC;
    cudaMalloc((void**)&MGA, size);
    cudaMalloc((void**)&MGB, size);
    cudaMalloc((void**)&MGC, size);

    cudaMemcpy(MGA, A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(MGB, B_transpo, size, cudaMemcpyHostToDevice);

    dim3 threadsPerBlock(nbr_thread);
    dim3 numBlocks((N * N + threadsPerBlock.x - 1) / threadsPerBlock.x);

    std::chrono::time_point<std::chrono::system_clock> start2, end2;
    start2 = std::chrono::system_clock::now();

    calcul << <numBlocks, threadsPerBlock >> > (MGA, MGB, MGC, N);

    cudaDeviceSynchronize();

    cudaMemcpy(C, MGC, size, cudaMemcpyDeviceToHost);

    end2 = std::chrono::system_clock::now();
    long long int microseconds2 = chrono::duration_cast<chrono::microseconds>(end2
- start2).count();

    end = std::chrono::system_clock::now();
    long long int microseconds = chrono::duration_cast<chrono::microseconds>(end -
start).count();


    cout << "Temps d'execution tout : " << microseconds << " micro sec\n";
    cout << "Temps d'execution calcul: " << microseconds2 << " micro sec\n";

    cudaFree(MGA);
    cudaFree(MGB);
    cudaFree(MGC);
```


Formations de Bacheliers et de Masters organisées par la Fédération Wallonie-Bruxelles
FÉDÉRATION WALLONIE-BRUXELLES
pôle hainuyer

```
    for (int i = 0; i < N; i++) {
        delete[] B[i];
    }
    delete[] A;
    delete[] B;
    delete[] C;
    delete[] B_transpo;

    return 0;
}
```